

DataCrumbs: Towards Explaining I/O Performance Variability in HPC Storage Stacks

Anonymized

Abstract

Modern scientific workloads exchange vast amounts of data across deep and complex I/O software stacks in high-performance computing (HPC) systems. Optimizing this data exchange requires a profound understanding of I/O performance across the entire I/O stack. State-of-the-art tools like Darshan capture performance counters that provide observability throughout the I/O stack. However, these tools capture only API-level counters that describe *what* happens, but fail to explain the observed behavior (i.e., the *why*). As a result, performance variation in similar I/O operations is often dismissed as system jitter without concrete evidence. To close this gap, we design DataCrumbs, a tool that extracts comprehensive performance data from the I/O stack to enable eXplainable I/O (XIO) analysis for I/O behavior in HPC systems. DataCrumbs leverages function-probing technology, offering greater flexibility, efficiency, and coverage than existing I/O performance tools. First, DataCrumbs flexibly attaches to multiple I/O software components, collecting events from over 8,723 functions with just 78 ms startup overhead. Second, DataCrumbs efficiently captures both user- and kernel-space functions with only 1% overhead, compared to Darshan DXT's 7%. Third, DataCrumbs collects performance data that reveals performance variation, inter-software stack interactions, and intra-software bottlenecks. In conclusion, by exposing the inner workings of I/O software layers, DataCrumbs enables explainable I/O (XIO) performance analysis in HPC storage systems.

Keywords

DataCrumbs, eBPF, kprobes, I/O performance, performance explainability, Multi-layer I/O monitoring, I/O stack

ACM Reference Format:

Anonymized. 2025. DataCrumbs: Towards Explaining I/O Performance Variability in HPC Storage Stacks. In *Proceedings of The International Conference for High Performance Computing, Networking, Storage, and Analysis (SC '25)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Data-intensive scientific workloads generate and analyze massive data volumes to derive insights, validate hypotheses, and achieve computational objectives on modern HPC systems. These workloads rely on diverse storage software solutions, backed by high-performance storage hardware, to support large-scale data exchange.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SC '25, St. Louis, MO

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

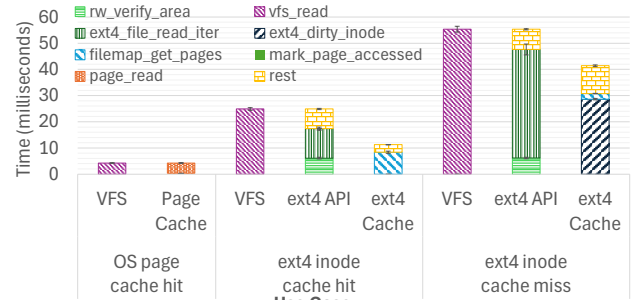


Figure 1: The experiment demonstrates the need for fine-grained monitoring of the kernel stack as the performance of similar read requests accessing different layers of the stack are 14× different. E.g., read operation time from OS page cache is 2ms, ext4 inode cache is 24ms, and device is 55ms.

Even with such infrastructure, I/O often accounts for more than 15% of runtime [32, 43, 46], motivating the need to optimize data access. Comprehensive understanding of I/O access patterns is key to improving efficiency and reducing time to solution [7, 14], making monitoring tools essential for accelerating scientific discovery.

HPC systems provide I/O monitoring tools that fall into two categories: *profilers* and *tracers*. Profilers [25, 47] collect coarse-grained statistics such as I/O counts, transfer size distributions, and per-file access patterns. Their low overhead (3–5%) makes them suitable for facility-wide deployment [6, 37, 39]. Tracers [15, 56, 58] capture every I/O event, enabling fine-grained bottleneck analysis, but introduce higher overhead (7–15%) [15]. Data from these tools feeds into analysis frameworks [7, 59] and research studies [14, 29, 33, 54] that identify problematic I/O patterns.

Existing I/O monitoring tools observe the data exchange behavior of scientific workflows by capturing user-space I/O APIs defined in *libc*. At the API level, I/O bottlenecks can be identified using performance-centric metrics such as I/O time and bandwidth. However, while these API-level events describe behavior (i.e., “what”) a workload is doing, they fail to explain the cause (i.e., “why”) certain performance behaviors occur. Understanding the cause typically requires an experimental approach, in which I/O experts hypothesize potential causes and design experiments to validate or refute them. Yet, many of these hypotheses are difficult to test on HPC systems due to restricted user-level access. For example, two identical read operations on the same device may differ significantly in latency, often attributed to system jitter, yet the true causes include page cache effects [16], ext4 inode cache effects [31], or actual jitter [38]. As Figure 1 shows, a 64 KB read may be served from the page cache in 4 ms, inode cache in 24 ms, or from the device in 55 ms, a 14× performance difference. Because such counters are inaccessible to user-space tools, identifying root causes requires profiling the deep

I/O software stack including application code, programming languages, runtime frameworks, I/O interfaces, middleware libraries, and kernel-space functions.

Another example arises in high-level libraries such as MPI-IO [50]. MPI-IO's two-phase I/O aggregates small per-process requests into fewer, larger writes per node, increasing effective transfer size but potentially limiting bandwidth if poorly tuned. Optimal performance requires tuning the number of aggregators to balance this trade-off. Current tools measure API-level writes but not internal routines like `dynamic_gen2` or `vulcan` [4], which implement two-phase I/O. Measuring these functions would help users select the right number of aggregators, but the lack of visibility forces costly trial-and-error tuning.

Together, these examples highlight the need for performance counters from both kernel-space and high-level I/O libraries. Visibility across all stages of the I/O stack allows users to detect when later stages are fast or bypassed, thereby revealing where bottlenecks truly originate and guiding data-driven optimizations.

Developing a comprehensive I/O monitoring tool capable of diagnosing the root causes of performance bottlenecks requires addressing three key challenges. First, the flexible discovery of software components and their routines, that includes the deep I/O stack, is a major obstacle in collecting data across all layers of the I/O software stack. Second, comprehensive data integration from multiple layers is crucial for effective root cause analysis, as isolated observations at individual layers provide limited insight. Finally, capturing thousands of functions introduces significant runtime overhead, as each function requires tracking and storing performance statistics. These challenges underscore the need for an I/O monitoring tool that automatically adapts to storage software and efficiently captures performance data from I/O software libraries and their internal functions. Such a tool would empower users to identify the root causes of I/O performance bottlenecks in HPC workloads with minimal overhead.

We introduce DataCrumbs, an I/O monitoring tool that provides a comprehensive view of the HPC storage stack to enable explainable I/O performance and address these challenges. DataCrumbs dynamically detects software routines using language-specific discovery methods (e.g., ELF analysis for C/C++ and DTrace hooks for Python) and inserts lightweight function probes to collect fine-grained data efficiently. Captured performance data is temporarily buffered in a kernel-level cache and asynchronously transferred to user space, decoupling capture from storage to minimize interference. By providing cross-layer performance data, DataCrumbs complements existing tools [7, 29] and enables root cause analysis of I/O bottlenecks. To achieve full-stack visibility across user- and kernel-space, the current implementation requires a one-time elevated access to set capabilities on the DataCrumbs executable; thereafter, users can run it entirely in user space without elevated privileges. We deploy the HPC software stack on a local HPC testbed to evaluate DataCrumbs's capabilities and validate its approach. The main contributions of this work are:

- (1) We propose *DataCrumbs*, a tool that captures fine-grained performance data from I/O software stack, enabling both observability and explainability of I/O systems.

- (2) We develop a *novel probing mechanism* for I/O monitoring that minimizes runtime overhead by decoupling performance data capture from storage.
- (3) We demonstrate *four exemplar use cases* from simulation and deep learning involving deep software stack of kernel caches, file system, MPI-IO and HDF5 to demonstrate how DataCrumbs's fine-grained performance data enables deeper insights for explaining I/O performance in HPC workloads.

2 Background

First we provide an overview of the eBPF functionality we leverage for low overhead metrics collection. Next, we delve into why the I/O stack is an extreme monitoring example that this approach can solve.

2.1 Extended Berkeley Packet Filter (eBPF)

Extended Berkeley Packet Filter (eBPF)[12, 20] is a Linux kernel technology that executes sandboxed programs in kernel space without modifying kernel code or loading modules. Widely adopted in cloud computing, it improves networking through efficient load balancing and traffic management[3, 23, 48], strengthens security with real-time monitoring and policy enforcement [11, 49], and offers low-overhead observability for troubleshooting distributed systems. In HPC, eBPF enables fine-grained monitoring of system calls, network events, and hardware activities for performance tuning and resource optimization, with JIT compilation ensuring efficient data collection [20]. By revealing when later I/O stages are fast or bypassed, eBPF can expose hidden bottlenecks across the stack. Tool developers use libraries such as BPF Compiler Collection (BCC)[53], BPFTrace[41], BPFTIME [60], and BPF Compile Once – Run Everywhere (BPF CO-RE) [36] to build eBPF programs with varying levels of abstraction. BCC provides Python/C++ bindings for kernel tracing, BPFTrace offers concise syntax for diagnostics, BPFTIME supports time-based profiling, and BPF CO-RE enhances portability. While low-level tools like raw BPF maximize control, higher-level tools improve usability with modest trade-offs.

Our work, DataCrumbs, builds on this ecosystem by enabling explainable I/O (XIO). Rather than relying on static configurations, DataCrumbs evaluates each application environment to deploy targeted eBPF hooks across libraries, middleware, and the kernel. Our contributions are threefold: (1) identify the right artifacts to monitor for XIO, (2) apply application-aware configurations to collect extensive metrics efficiently, and (3) analyze these metrics to explain when and why bottlenecks occur. This stage-aware approach provides actionable insights beyond observability, allowing root-cause diagnosis and optimization of HPC I/O performance.

2.2 Complexity in I/O software stack

The I/O software stack in HPC systems is complex and deeply layered, spanning from low-level interfaces to high-level abstractions including spanning multiple nodes and often multiple networks. At its foundation, HPC applications interact with basic I/O interfaces such as POSIX and STDIO, which define fundamental read/write operations. These interfaces serve as the backbone for higher-level I/O libraries like HDF5 [51], PnetCDF [28], and ADIOS [30], which provide structured access to scientific datasets. Sitting in-between

are middleware solutions such as UnifyFS [9] and Hermes [26], which optimize data placement, caching, and prefetching. Beneath the middleware, kernel file systems like ext4 and XFS handle local disk storage, while large-scale global file systems such as IBM Spectrum Scale (GPFS) [44] and Lustre [8] orchestrate parallel I/O across distributed storage nodes. This deep software stack introduces substantial complexity in managing and optimizing I/O performance.

A key challenge in I/O performance tuning is the interdependence between software layers, where changes in one can significantly affect others [13, 22]. For example, an application using PnetCDF depends on MPI-IO for distributed access, which in turn relies on the POSIX layer and the underlying parallel file system. PnetCDF's metadata and data layout choices can influence MPI-IO's behavior, which then affects how data is ultimately written. Similarly, configurations at the POSIX layer, such as file locking and consistency semantics, can impact both middleware and storage performance [55]. Achieving optimal performance requires coordinated tuning across these layers. A change in MPI-IO's aggregation strategy, for instance, might require adjustments to file system stripping to prevent contention and inefficient access patterns [50].

In addition to inter-layer dependencies, the internal complexity of I/O libraries further complicates performance explainability. HDF5, for example, includes multiple components such as metadata management, dataset chunking, and collective I/O, each of which affects how data is written to storage [13]. Understanding the impact of these components is essential for effective tuning. MPI-IO's ROMIO implementation also provides tunable parameters that control data aggregation [4]. Adjusting the number of aggregators involves balancing the overhead of aggregation with potential bandwidth improvements. Too few aggregators may lead to poor storage utilization, while too many can increase synchronization overhead and degrade performance. These trade-offs emphasize the need for comprehensive performance data to support explainable I/O analysis and guide tuning decisions across the entire I/O stack. Crucially, such visibility must extend beyond a single node: because these libraries operate collectively across many nodes and networks, instrumentation at scale is required to capture the full picture of I/O behavior.

3 Design

We introduce DataCrumbs, an I/O monitoring tool designed to collect detailed performance data from the entire I/O stack, enabling a deeper understanding of observed I/O behaviors in workloads. By providing the observation and its cause, DataCrumbs eliminates the need for application developers and researchers to manually navigate the complex optimization search space through experimentation. Our approach is guided by three core objectives:

- (1) supporting diverse software layers by dynamically extracting functions across different toolchains,
- (2) maintaining low overhead to ensure minimal impact on workload performance,
- (3) optimizing collection for efficient large-scale trace analysis.

At a high level, DataCrumbs consists of seven core components that capture, process, and store performance data in HPC workloads (Figure 2). Upon job allocation, DataCrumbs initializes across all nodes to monitor the storage software stack. It detects loaded

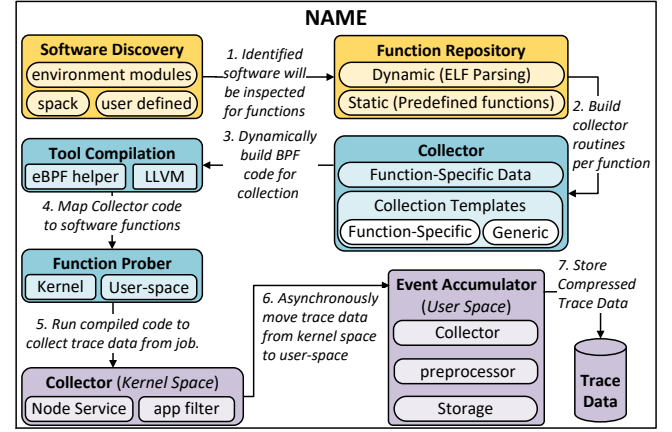


Figure 2: DataCrumbs's high-level design for flexible, efficient, and comprehensive performance data collection in HPC storage software stacks.

software through environment modules and Spack [18], populating a collection catalog that can be customized with user-defined configurations. Once identified, DataCrumbs extracts functions from the software (Step 1) using strategies like parsing Executable and Linkable Format (ELF) files and predefined Python hooks. It then builds function-specific collector routines (Step 2), capturing function arguments for tracing and high-level statistics for profiling. To optimize collection, predefined templates minimize the number of probed functions. The collector functions are compiled into kernel code using LLVM (Step 3), leveraging predefined helpers to capture performance data at function entry and exit. This kernel code is implemented in BPF using the BPF CO-RE toolkit. DataCrumbs then attaches probes at function boundaries (Step 4) using k-probes for kernel space and uprobes for user space.

Once deployed, DataCrumbs runs as a kernel-space service (Step 5), capturing data from all processes on the node. To enable targeted monitoring, a lightweight library provides two custom functions that identify processes of interest, ensuring only relevant application or system software is profiled. During workload execution, DataCrumbs collects performance data and stores it in kernel space, using maps for profiling mode and an event ring buffer for tracing mode. A user-space thread asynchronously reads the performance data from the kernel space, processes it, and writes it to the filesystem (Step 6). When the workload completes, DataCrumbs waits for a configurable period before compressing the collected trace and terminating (Step 7). This workflow enables efficient, low-overhead monitoring while remaining adaptable to different workloads.

3.1 Hooking into Deep I/O Software Stacks

In HPC environments, understanding software behavior is crucial for optimizing performance and diagnosing inefficiencies. However, manually identifying relevant software components and extracting function-level details can be complex and time-consuming due to the diverse range of software stacks and programming models used in HPC workloads. To address this, DataCrumbs provides an automated approach for discovering software and extracting

function information. It dynamically detects loaded software using environment modules, Spack environments, and user-defined inputs, ensuring comprehensive coverage of system and application libraries. Once identified, DataCrumbs automatically extracts function details using a combination of predefined system calls, ELF symbol analysis for C/C++ binaries, and runtime hooks for interpreted and GPU-based languages like Python, CUDA, and HIP. This automation enables efficient and scalable monitoring, reducing the manual effort required to profile complex HPC workloads.

DataCrumbs automatically detects software using three primary methods: environment variables, Spack environments, and user-defined inputs. First, DataCrumbs leverages environment variables to identify loaded software modules. It runs the `module list` command to obtain a list of active modules and then uses `module whatis` to retrieve metadata about each module, including environment variables it defines. Among these, DataCrumbs extracts the library path variable, which typically points to shared object (.so) files. It then uses the `find` utility to locate relevant .so files within these directories, ensuring comprehensive software detection. Second, DataCrumbs integrates with Spack environments to discover installed software packages. It executes `spack find` to list installed packages and, for each detected package, inspects its library folder to extract the associated shared object (.so) files. This approach ensures compatibility with Spack-managed software stacks, which are commonly used in HPC environments. Finally, DataCrumbs provides a user-defined option, allowing users to explicitly specify a list of object files for parsing. This flexibility ensures that even software not detected through environment variables or Spack can be manually included for monitoring. By combining these three methods, DataCrumbs ensures a robust and adaptable approach to software discovery.

DataCrumbs automatically extracts function information from software using three primary methods, depending on the type of functions being identified: predefined functions, C/C++ executables and libraries, and interpreted/runtime functions like Python. First, DataCrumbs includes a set of predefined functions commonly associated with system-level I/O operations. These include system calls related to storage, such as those in the Virtual File System (VFS), OS caching mechanisms, various file systems like XFS, ZFS, and ext4, and the Block I/O layer. Additionally, DataCrumbs tracks standard libc I/O functions such as POSIX and STDIO APIs, which are frequently used by HPC applications for file operations. These predefined functions allow DataCrumbs to capture critical I/O behavior without requiring additional discovery steps. For C/C++ executables and shared libraries, DataCrumbs uses ELF symbols to extract function names. This method relies on debugging symbols embedded within binaries to accurately describe function details. If debugging symbols are present, DataCrumbs can retrieve precise function signatures and locations, enabling efficient monitoring of compiled software components. For Python codes, DataCrumbs uses function entry and exit hooks provided by the interpreter to capture workload-specific functions. This hook has to be enabled during the building of Python software.

By combining these approaches, DataCrumbs provides a comprehensive mechanism to automatically discover I/O software and track function execution across diverse stacks, ensuring detailed performance monitoring. This cross-layer integration overcomes

the limitations of isolated observations and delivers the full context needed for effective root cause analysis.

3.2 Efficient Decoupled Runtime Collection

Collecting performance data at runtime is essential for understanding system behavior in HPC environments, but traditional tracing approaches often introduce significant overhead, especially as the volume of collected events increases. To address this, we need a decoupled performance data collection mechanism that minimizes performance impact while handling large-scale data efficiently. Instead of synchronously processing each collected event—an approach that scales poorly—our method decouples data capture from processing and storage. By immediately capturing relevant function calls and storing them in lightweight kernel-space in-memory buffers, we enable asynchronous processing, where data is analyzed and written to storage independently. This design significantly reduces runtime overhead, ensuring that system performance remains stable and responsive even as the number of traced events grows.

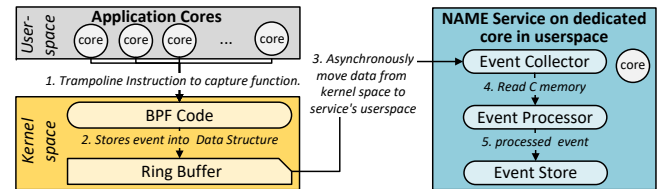


Figure 3: High-level overview of DataCrumbs's decoupled runtime, enabling function tracing with low overhead.

Figure 3 illustrates the high-level workflow of this decoupled runtime collection mechanism. DataCrumbs begins by attaching jump instructions, known as trampolines, at the assembly level to capture function call boundaries (Step 1). This low-level probing mechanism is inherently language-agnostic since all programming languages ultimately compile down to assembly instructions for execution. Once function events are captured, they are stored in highly efficient kernel-level data structures such as Maps and Ring Buffers (Step 2). These structures are designed for low-cost insertions and lookups, making them ideal for high-frequency, temporary data storage. Instead of synchronously processing captured events, DataCrumbs employs a user-space processing engine that asynchronously reads data from these kernel-level structures (Step 3). This decoupling ensures that the overhead of processing and storing data does not impact the application's execution. In this case, our processing rate needs to be faster than collection rate and this mitigates buffer overflow cases. Additionally, even if an application is terminated, DataCrumbs continues to process residual kernel space data without interruption. Once the data is transferred to user space, it is first converted from raw byte format into structured C data types (Step 4). It then undergoes trace resolution and optimization for efficient storage (Step 5) before being written to a per-node trace file (Step 6). By separating data capture from processing and storage, DataCrumbs minimizes runtime overhead while enabling scalable and efficient performance monitoring in HPC systems.

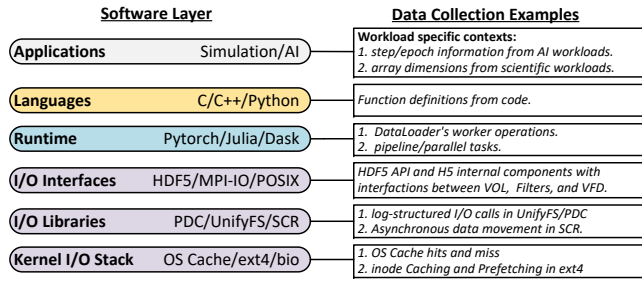


Figure 4: DataCrumbs capturing multi-layer performance data in HPC storage systems

This design directly supports our contribution by minimizing runtime overhead through the decoupling of performance data capture from storage, ensuring that monitoring remains lightweight while providing high-fidelity insights.

3.3 Performance Data Collection

We focus on three key aspects to achieve comprehensive performance data collection in HPC storage systems. First, we capture performance data across multiple layers, including system I/O calls, file systems, and the Block I/O layer. Second, we utilize two collection modes: profiler, which provides aggregated statistics with low overhead, and tracer, which records fine-grained event details. Finally, we design an efficient trace format optimized for compact storage and fast retrieval. In the following sections, we describe these components in detail, highlighting their role in scalable and efficient performance monitoring.

3.3.1 Capturing performance data from multiple layers. DataCrumbs enables comprehensive performance data collection across multiple layers of the HPC storage software stack, as illustrated in Figure 4. The key challenge for collecting this data is determining the most relevant data for understanding I/O performance, which varies by layer and software context. The figure presents a typical HPC storage stack, covering application layers, programming languages, runtimes, I/O interfaces, middleware libraries, and the kernel stack.

Each layer provides unique insights. At the application level, we capture function execution data and workload-specific contextual information, such as scientific data dimensions, training steps in AI models, or simulation iterations, which are critical for diagnosing performance bottlenecks. At the language level, we extract function signatures, parameters, dependencies, and coverage to analyze function impact across layers. The runtime level provides execution-specific details; for example, in PyTorch, we can track data loading efficiency, while in task-based systems like Julia and Dask, we capture metrics that track task pipelining and parallelism. At the I/O interface level, we track interactions between software components, such as how HDF5 calls MPI-IO, which in turn invokes POSIX, allowing us to analyze how optimizations like chunking, filters, and data type conversions affect lower layers and overall storage performance. Additionally, monitoring function calls within I/O libraries like UnifyFS [9], SCR [34], and PDC [10] provide insights into how effectively they optimize application I/O and whether they

behave as expected. Finally, at the kernel I/O stack, we capture OS cache hit/miss rates, ext4 prefetching, and block I/O performance, enabling a detailed assessment of system-level optimizations.

By analyzing each layer individually and in relation to others, we gain both fine-grained system insights and high-level workload-specific context, which together enable a holistic understanding of I/O performance in HPC environments. This layered capture of performance data not only ensures observability but also provides the causal links needed for explainability of I/O system behaviors.

3.3.2 Profiler and Tracer Mode. Different types of performance analysis require varying levels of data granularity. Some use cases, such as understanding the overall contribution of a function to system performance, benefit from aggregated high-level statistics, while others, like modeling function hierarchies and dependencies, require fine-grained tracing. To support both scenarios, DataCrumbs offers two distinct modes: profiler and tracer, allowing users to adjust data collection granularity based on their analysis needs.

In profiler mode, DataCrumbs collects event data such as time elapsed and function call frequency, storing this information in per-time-interval and per-process maps. This mode is efficient and incurs an overhead of about 0.1% on the application runtime for temporary storage, making it ideal for monitoring system-wide performance with minimal overhead. In contrast, tracer mode records detailed event-level information and function call sequences in a ring buffer, enabling precise modeling of function execution but incurring greater overhead (approximately 1%). These overheads are collected by measuring the runtime of workloads with and without running DataCrumbs.

Choosing between these modes depends on the analysis objective. Tracing is essential for exploratory analysis, where identifying critical functions and their exact impact is necessary. In contrast, profiling provides a broader view of workload behavior, making it well-suited for continuous system monitoring. Given this flexibility, DataCrumbs can also be deployed as a system-wide profiler with selective tracing enabled for specific workloads.

3.3.3 Trace format to handle performance data. Capturing comprehensive performance data in HPC storage systems generates an enormous volume of trace data—potentially 1000× more than state-of-the-art tools like Darshan. Efficiently storing and analyzing this data requires a well-designed trace format that minimizes overhead while enabling fast retrieval and scalable analysis. To achieve this, our trace format is designed around three key objectives:

- (1) **Buffered Writing:** As trace data is written at high frequency, we use buffered writing to efficiently flush data to disk in batches, improving write performance.
- (2) **No Event Dependencies:** Each event is stored independently, without ordering or dependency constraints, allowing for embarrassingly parallel processing.
- (3) **Analysis-Friendly Structure:** The trace format is optimized for parallel parsing, enabling scalable analysis using tools like Dask [42]. This ensures that large-scale performance data can be processed efficiently without bottlenecks.

DataCrumbs builds on the DFTracer trace format [15], an efficient and analysis-friendly representation designed for large-scale

performance data collection. Our trace format follows the JSON Lines structure, ensuring portability and structured data storage while enabling efficient parsing and querying. To optimize this format for DataCrumbs, we introduce three key extensions. First, we implement string hash management to reduce trace size. Instead of storing full strings (e.g., filenames) in every event, we replace them with consistent hash identifiers and introduce string hash events that map these hashes to actual strings. This significantly minimizes redundancy while maintaining efficient lookups. Second, we add metadata events to capture workload-specific context, such as training steps, epochs, or simulation iterations. These contextual details are essential for understanding workload behavior but are not inherently part of function call traces. Finally, we introduce counter events, which store per-function, per-process statistics, allowing aggregated profiling data to complement detailed tracing without excessive storage overhead.

Beyond these enhancements, DataCrumbs retains key features of DFTracer, including gzip compression with indexing, which enables fast querying and parallelized trace loading, ensuring that the format remains efficient and scalable, supporting high-performance comprehensive performance analysis.

3.4 Performing Explainable I/O Analysis

Explaining I/O performance is essential to apply the most effective optimizations in HPC workloads. DataCrumbs collects extensive performance data, but meaningful insights require efficient parsing, scoping, and analyzing the performance data. To achieve this, we need two key capabilities: a) an efficient analysis pipeline that can process large-scale performance data at scale and b) an event association technique to link disjoint but related events. The analysis pipeline is similar to the one described in DFTracer’s analysis pipeline [15]. Once the data is loaded, we need to associate events to enable explainable I/O performance analysis through DataCrumbs.

Understanding I/O performance requires linking relevant events across different layers of the HPC storage stack. We can analyze DataCrumbs’s performance data using Dask DataFrame [42], first filtering key events at the API layer and then identifying all associated events. However, performing this analysis on billions of events is computationally expensive ($O(n^2)$ where n is the number of events) as we need to identify if an event have any overlapping events in that thread. To address this, we introduce a novel time-based parallelization algorithm that efficiently detects overlapping events, reducing computational overhead while preserving accuracy. This approach enables scalable and efficient root cause analysis of I/O behavior in complex workloads.

Figure 5 illustrates the product of our time-based event association algorithm, which efficiently links related events for large-scale I/O performance analysis. This algorithm operates under the assumption that events and their underlying causes exhibit a short, windowed temporal correlation, allowing related events to be grouped based on their proximity in time. In cases where asynchronous events introduce long delays between cause and effect, the algorithm must be adjusted to account for extended temporal windows to ensure accurate association. The algorithm begins by marking API interface calls that need explanation (Step 1). For each API call, we identify its time range per process and find all events

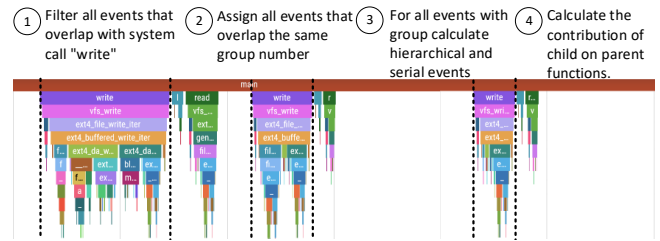


Figure 5: The associative algorithm identifies target API’s related events, enabling explainable I/O analysis.

that fall within this range. These events are then grouped and assigned a unique group number to facilitate parallelized querying during analysis (Step 2). This grouping allows us to establish associations between related events, enabling the calculation of event hierarchies and serialization (Step 3). Finally, once the relationships between all events are identified, we compute the contribution of each captured event to the targeted API call, providing insights into its performance impact (step 4). This reduces the time complexity by the number of time intervals (i.e., $O((n/t)^2)$, where t is the number of time ranges). Additionally, each of t event groups can be embarrassingly parallelized on distributed Dask workers.

This analysis links performance data from DataCrumbs across multiple software layers, enabling precise identification of I/O performance bottlenecks and their underlying causes. By correlating events from user-level libraries down through the kernel and file systems, the approach exposes cross-layer interactions that are often hidden in isolated views. Our time-based parallelization algorithm ensures scalability, making large-scale analysis computationally feasible. Together, these capabilities enhance explainability by revealing not just where performance issues occur, but why they emerge, thereby equipping researchers and developers with actionable insights to optimize I/O performance more effectively.

4 Evaluation

This section comprehensively analyzes DataCrumbs’s flexibility, performance, and coverage through several key evaluations. First, we demonstrate DataCrumbs’s ability to dynamically discover and instrument thousands of routines across the deep I/O software stack, addressing the challenge of identifying and monitoring diverse software components. Next, we quantify DataCrumbs’s runtime overhead to evaluate its efficiency in capturing extensive performance statistics, directly tackling the challenge of managing the cost of monitoring thousands of functions. Finally, we present use cases that highlight DataCrumbs’s ability to integrate data across multiple layers, enabling comprehensive root cause analysis and overcoming the limitations of isolated observations.

4.1 Methodology

This subsection details the testbed, software tools, and workloads used in the evaluation.

4.1.1 Testbed. All experiments were conducted on a local test bed. Each compute node features dual Intel(R) Xeon(R) E5-2667 CPU (24 cores total), 128 GB RAM, 10GbE, and a 512GB local SSD. The

Table 1: DataCrumbs’s probe-attachment overhead grows with the number of instrumented functions, but the growth is sublinear, with diminishing per-function cost at higher layers.

	Selective I/O Calls	All POSIX + Kernel	MPI	HDF5
Runtime (ms)	0.448	24.321	33.041	78.980
Number of APIs	100	2489	3544	8723

system runs the TOSS 5 operating system, a variant of Redhat Linux 9 with 5.14.0 kernel version.

4.1.2 Software. DataCrumbs uses BPF CO-RE [1] (v0.30.0) built on eBPF (Kernel 5.6) and requires CMake 3.4 for compilation and Python 3.10 for execution. The setup also includes GCC 11.4.0, OpenMPI 5.0.5, and HDF5 1.14.5. For trace analysis, we use Dask, and Perfetto UI for visualization.

4.1.3 Workloads. All experiments use the IOR benchmark [27] and DLIO Benchmark. The IOR test was conducted with the HDF5 interface on a single node with 24 processes, using a file-per-process access pattern with sequential access. Each process wrote 1 GB blocks with a 1 MB transfer size. This configuration was chosen to generate a sustained stream of I/O events without pauses or interference from other application activity, ensuring stress on the monitoring infrastructure. We execute the Unet3D workload within the DLIO benchmark running on 4 nodes with 4 processes per node. We stage-in the dataset into the local SSD before training. Each test runs ten times, and the figures report the mean and standard deviation. We compare DataCrumbs against Darshan, the most widely deployed HPC I/O monitoring tool, for performance and overhead comparisons. We measured other tools like Score-P and Recorder, but the level of information was similar to Darshan with more overheads on the workload.

4.1.4 Performance Data. As introduced in Section 3.3, DataCrumbs supports two modes: tracer and profiler. Tracer mode captures individual events as traces for visualizing hierarchical software stack dependencies via Perfetto UI. Profiler mode aggregates data by time interval, process, and I/O function, offering insights into each layer’s contribution through function-level counters. This aggregated view helps explain I/O performance variability in applications.

4.2 Flexibility to handle the Deep I/O Software Stack

The IOR test (refer to Section 4.1.3) uses the HDF5 interface. The goal of the experiment was to evaluate the runtime cost of attaching function probes at different software layers, ranging from selective I/O calls to higher-level libraries like MPI and HDF5. The DataCrumbs tool was dynamically configured to profile different software layers, with functions discovered using the ELF format of the I/O libraries.

The evaluation uses two key metrics: (1) the runtime to attach function probes and (2) the number of functions profiled across four configurations (shown in Table 1). In each test, we include the functions of the previous tests. *Selective I/O Calls*, which include 100 carefully chosen system calls and kernel functions, required only 448 μ s to attach probes. Expanding to *All POSIX + Kernel* functions

Table 2: Profiling overhead: DataCrumbs captures 1000x more events with lower overhead than Darshan, with typical variability < 1%.

# I/O Events	Base (sec)	Darshan (sec)	DataCrumbs (sec)
10K	0.252s+-0.001	0.259s+-0.009	0.252s+-0.004
100K	2.419s+-0.033	2.492s+-0.047	2.422s+-0.067
1M	23.255s+-0.52	23.952s+-0.61	23.278s+-0.52

(2489 functions) increased the overhead to 24 ms, showing that capturing a broader set of system interactions introduces additional costs. Adding MPI-IO functions (3544 total) further raised the overhead to 33 ms, while the most extensive configuration, HDF5 (8723 functions), incurred the highest overhead at 78.9 ms.

While the overhead increases as more functions are profiled, the growth is nonlinear, with diminishing per-function costs at higher layers. This suggests that although profiling additional software layers incurs overhead, the relative impact decreases as the function set expands. More importantly, in the context of HPC workloads that typically run for hours, the total overhead of attaching probes, measured in milliseconds, is negligible compared to overall job execution time. An important finding is that manually identifying and instrumenting these functions is not feasible, given the sheer number of functions across different software layers. This highlights a key advantage of DataCrumbs, which automates function discovery and instrumentation, allowing scalable and efficient profiling without requiring manual intervention. These findings demonstrate that fine-grained function tracing is practical, enabling deeper insights into I/O behavior with minimal impact on performance.

4.3 Overhead on Application Runtime

The IOR test was also configured to generate 10K, 100K, and 1M I/O events by adjusting the block size. Two tests were conducted: one in profiling mode and another in tracing mode, comparing the overhead of DataCrumbs against Darshan (for profiling) and Darshan Extended Tracing (DXT) (for tracing).

Table 2 compares the application runtime under different profiling solutions. Across all event sizes, DataCrumbs incurs minimal overhead, closely matching the base runtime without profiling, while Darshan adds a slight increase (e.g., 0.492s additional overhead for 1M events). Despite this, DataCrumbs captures significantly more events, 125M+ for 100K I/O events and 1.25B+ for 1M I/O events, compared to Darshan’s fixed one-to-one capture. Notably, the profiling overhead remains below 0.1%, making it negligible for long-running HPC applications. We also evaluated Score-P where the overhead of capturing profiles was 4% larger than Darshan by 33%.

Table 3 evaluates tracing overhead, where DataCrumbs again remains close to the base runtime. For example, at 100K events, Darshan DXT increases execution time to 2.624s, while DataCrumbs remains lower at 2.489s, despite capturing over 125M events. The trend persists at 1M events, where DataCrumbs captures 1.25B+ events while incurring a marginal overhead of 0.428s over the base. The maximum tracing overhead is just 1%, demonstrating efficient event collection with minimal performance impact. We also evaluated DFTracer, where the overhead for capturing events was 7%, which makes DFTracer overhead 7 $\times$ more than DataCrumbs.

Table 3: Tracing overhead: DataCrumbs captures 1000x more events with lower overhead than Darshan, with typical variability < 1%.

# I/O Events	Base (sec)	Darshan DXT (sec)	DataCrumbs (sec)
10K	0.252s±0.001	0.269s±0.01	0.252s±0.004
100K	2.419s±0.033	2.624s±0.047	2.489s±0.067
1M	23.255s±0.52	25.350s±0.61	23.683s±0.52

A key finding is that DataCrumbs captures orders of magnitude more events than Darshan, yet its overhead remains comparable or lower. This efficiency is due to two key design choices: (1) decoupling event capture from storage, ensuring that the application is not blocked for storing events, and (2) performing event capture in the kernel space, which is highly efficient compared to user-space instrumentation. Additionally, DataCrumbs dynamically instruments 8723 functions across Kernel, POSIX, MPI, and HDF5, providing a more comprehensive view of I/O behavior. These results highlight that DataCrumbs enables high-fidelity I/O analysis with minimal runtime impact, making it a scalable and effective alternative to existing profiling and tracing tools.

4.4 XIO: Use Case Study

DataCrumbs supports a range of I/O analysis use cases, providing deeper insights into performance bottlenecks. This section explores four key application areas: (1) explaining performance variability in I/O operations by exploring kernel functions, (2) analyzing performance differences between I/O paths, such as Direct I/O and fsync on write, to determine the most efficient approach, (3) breaking down I/O performance across multiple software layers, from kernel calls to high-level I/O libraries, and (4) explain I/O performance of deep learning workload using deep software stack.

4.4.1 Performance Variability in I/O Operations. In the IOR test described in Sec. 4.1, we switch the interface to POSIX and examine two write operations that exhibit a 40% performance variation. Using DataCrumbs’s profiling, the test captured detailed insights across the I/O stack, including interactions at the libc interface and the ext4 filesystem layer on SSD storage.

Figure 6 shows the runtime breakdown of these two operations, highlighting the differences in their execution. Despite the overall 39.6% performance gap, the costs of important ext4 file system functions, `ext4_da_write_begin` (34.6ms vs. 33ms) and `ext4_da_write_end` (38.4ms vs. 36.3ms), remain nearly identical across both cases. The primary factor contributing to the discrepancy is inode buffering behavior. In the first case, `file_modified` writes to an existing inode page, while in the second, it allocates a new inode page, introducing additional overhead. This result demonstrates how seemingly identical I/O operations can experience significant performance differences due to underlying filesystem behavior. The ability of DataCrumbs to break down execution across multiple layers enables pinpointing such inefficiencies, offering valuable insights for optimizing I/O performance.

4.4.2 Analyzing Performance Differences between I/O Paths. In this study, we used IOR with the POSIX interface again. The benchmark was executed under two different configurations: buffered writes with fsync enabled and Direct I/O writes. Using the profiler mode, DataCrumbs captured the execution information spanning



Figure 6: DataCrumbs captures all layers to allow analysis of performance variability in identical I/O operations by revealing differences in inode buffering.

multiple layers, including libc (user-space I/O interface), ext4 (filesystem layer), and bio (the kernel block I/O layer for accessing block devices).

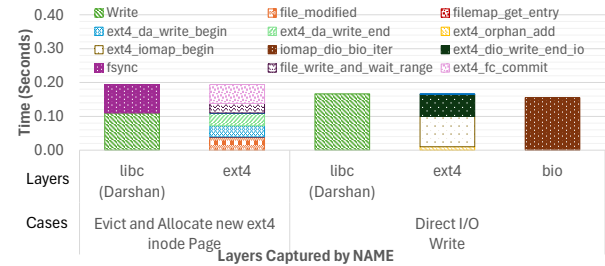


Figure 7: DataCrumbs captures all layers and reveals the trade-offs between buffered and Direct I/O writes. While buffered writes are faster, fsync introduces significant overhead, making Direct I/O more efficient for data persistence.

Figure 7 presents the runtime for both cases, revealing that buffered writes are initially 50% faster than Direct I/O due to in-memory caching in the ext4 layer. However, this advantage is offset by fsync, which forces data to be written to disk, making the overall buffered write with fsync 16% slower than Direct I/O. A detailed breakdown at the ext4 layer highlights key differences in function calls. Buffered writes rely on `file_modified` (35% of execution time) and ext4 write calls (i.e., `ext4_da_write_begin`/`ext4_da_write_end` are 65% of the execution time), while fsync incurs additional overhead, dominated by `file_write_and_wait_range` (33% of fsync time) and `ext4_fc_commit` (67% of fsync time). In contrast, Direct I/O bypasses caching, leading to different ext4 file system function calls in the kernel: `ext4_orphan_add` (6%), `ext4_iomap_begin` (53%), and `ext4_dio_write_end_io` (39%), with kernel block I/O layer operations contributing to 93% of ext4’s execution time. These results highlight the trade-offs between buffered and Direct I/O writes. While buffered writes are faster, the need for fsync introduces significant overhead, making Direct I/O more efficient in cases where immediate persistence is required. By capturing detailed execution across the I/O stack, DataCrumbs provides valuable insights into I/O performance variations, enabling informed decisions on optimizing file system interactions.

4.4.3 Performance Analysis Break-down for Stacked I/O Software. IOR was run with DataCrumbs profiling mode, capturing execution across I/O stack, including Kernel, POSIX, MPI-IO, and HDF5. This

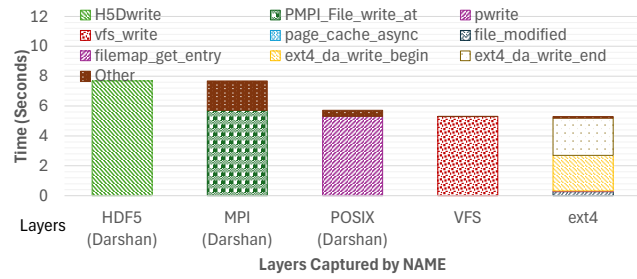


Figure 8: DataCrumbs breaks down execution across the entire I/O stack, identifies contribution of different layers and pinpoints that ext4 needs tuning.

use case focuses on breaking down the H5Dwrite operation, a key function in HDF5-based I/O workflows, to identify contribution of each stack to runtime.

Figure 8 presents the runtime breakdown of H5Dwrite across different layers. The total H5Dwrite operation takes 7.68 seconds, with the majority of the time spent in lower layers. PMPI_File_write_at, the MPI-IO function, accounts for 5.317 seconds, and within that, pwrite (POSIX layer) takes the whole time. At the ext4 layer, the execution further breaks down into ext4 buffering through file_modified (0.27s) and writing data into ext4 buffers using ext4_da_write_begin (2.38s), and ext4_da_write_end (2.50s). This detailed breakdown reveals that most of the time is spent in the ext4 layer, indicating that optimizing HDF5, MPI-IO, or POSIX layers would have minimal impact on overall performance.

This result highlights a key advantage of DataCrumbs: its ability to break down execution across the entire I/O stack, allowing users to pinpoint where optimizations are needed. In this case, improvements should focus on ext4 filesystem behavior, rather than higher-level I/O interfaces. By providing such fine-grained insights, DataCrumbs helps guide targeted optimizations, improving efficiency in complex I/O workloads.

4.4.4 Explaining I/O Performance of a Deep Learning Workload.

The Unet3D workload targets volumetric segmentation in medical imaging, leveraging a 20-layer deep convolutional network trained on 2,500 sparsely annotated images (140 MB each, totaling 353 GB in NPZ format)[24]. Using PyTorch for data-parallel training, the workload processes batches of four images across six threads per GPU, running on four nodes with four GPUs each for twenty epochs without checkpointing. Figure 9 illustrates the time spent (y-axis, in seconds) across different software layers (x-axis) in the I/O stack while running this workload under DataCrumbs instrumentation. The main process executes computation interleaved with I/O wait, while worker threads execute DataLoader routines that call NumPy for sample loading, which in turn issues POSIX system calls. These calls propagate through the VFS and OS cache, and cache misses result in Lustre file system accesses.

We observe that the main process spends only about 10% of its execution time waiting for I/O. This indicates that the majority of I/O stalls are effectively overlapped with computation, minimizing the impact of I/O latency on overall training time. The bulk of the time attributed to I/O processing occurs outside the main process,

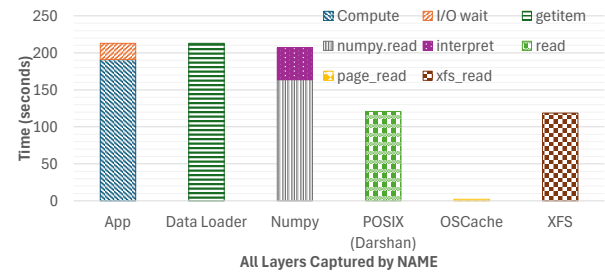


Figure 9: DataCrumbs extracts all performance data from all layers to explain performance characteristics of deep learning workload.

specifically within the worker threads that handle data loading and preprocessing.

The DataLoader layer spends a total of 212 seconds handling data samples, achieving an overall bandwidth of 32 GiB/s. Out of this total time, only 10% cannot overlap with computation, indicating that asynchronous prefetching in the DataLoader is largely effective. The analysis also reveals that NumPy operations dominate the data loading phase, contributing 97% of the time cost, with 76% (163 seconds) spent on actual I/O and the remainder on interpreting the binary data into Python arrays.

At the system call level, the POSIX interface accounts for 120 seconds of the data reading cost, corresponding to an effective bandwidth of 56 GiB/s. This shows that the underlying system is capable of high-performance I/O, but part of the bandwidth is lost due to Python-level overhead in NumPy's data interpretation. These observations highlight the gap between raw filesystem throughput and the effective bandwidth available to the deep learning framework.

Within the filesystem layer, 80% of the data is served directly from the OS cache, yielding a bandwidth of 33 GiB/s, while the remaining 20% of the data is fetched from XFS at a significantly lower rate of 144 MiB/s. This high cache hit rate demonstrates that the workload benefits from in-memory caching and does not require additional optimization.

The evaluation highlights that DataCrumbs capability to capture synchronized and asynchronous I/O activities across multiple software layers, including Python-based libraries and C++ system calls, and across the full filesystem stack. This comprehensive visibility explains the achieved bandwidth values and validates that the observed performance is well-suited to the tested system, eliminating the need for further I/O optimizations in this case.

5 Related Work

We review both traditional and modern monitoring tools that support I/O performance analysis and optimization in HPC.

5.1 Traditional HPC Monitoring Tools

Traditional monitoring tools have provided insights into system performance by focusing on collecting and visualizing metrics such as I/O bandwidth, IOPs, and file-specific counters, aiding administrators and researchers in maintaining system health and optimizing performance.

With regard to system monitoring tools, Lustre Monitoring Tool [19] (LMT), mmpmon [21], and FSMonitor [40] are widely utilized in HPC environments. LMT is tailored for Lustre-based filesystems, providing real-time and historical visualization of server activity metrics by monitoring /proc and /sys interfaces and storing data in a MySQL database. mmpmon collects I/O statistics in GPFS to monitor application I/O demand, track usage patterns, analyze node load distribution, and record application request service times over extended periods. FSMonitor is a scalable, file-system-independent monitoring tool that uses a modular Data Storage Interface (DSI) architecture to efficiently detect and report events across various local and distributed file systems. It has demonstrated the capability to process nearly 38k events per second on Lustre. While effective within their specific domains, these tools primarily focus on collecting system-level counters to detect bottlenecks but do not provide insights into the underlying causes of performance issues.

In the scope of application-level tools, Darshan [45], Score-P [25], Recorder [57], and DFTracer [15] are tools that provide profiling and tracing capabilities for specific HPC applications. Darshan is a lightweight tool that captures I/O behavior by intercepting calls to APIs such as HDF5, MPI, POSIX, and STDIO, offering per-layer statistics on access patterns. Score-P is a performance measurement infrastructure supporting profiling and event tracing across various programming paradigms, capturing performance data across multiple software stack layers. Recorder is a parallel I/O tracing tool that records detailed information about I/O operations across various layers, capturing function parameters for each I/O call. Among the three, Darshan imposes the least runtime-overhead on the application. DFTracer is tailored for AI-driven workflows and captures data-centric events across multiple software stack layers, including workflows and I/O middleware. However, these solutions are limited to user-space data collection and cannot delve into kernel-space events. This limited user-level access restricts their ability to correlate data across different layers or explain performance anomalies from both user and kernel software stacks.

5.2 Emerging eBPF-based Monitoring Tools

The complexity of I/O operations in HPC applications necessitates advanced monitoring tools capable of providing deep insights into system behavior. Traditional tools have offered foundational capabilities but often lack the granularity required for comprehensive analysis. Emerging technologies, particularly those leveraging the Extended Berkeley Packet Filter (eBPF), present innovative solutions to these challenges. We look at the following three key tools using eBPF.

EZIOTracer [35] unifies kernel and user-space I/O tracing for data-intensive applications using eBPF. It employs the Open Trace Format 2 (OTF2) [17] and has been evaluated with tools like Flexible I/O (FIO) [2]. However, EZIOTracer primarily compares its overhead to predecessors like EzTrace [52] and does not focus on multi-level performance explainability or internal function tracing, concentrating instead on public APIs. Tetragon [49] is an eBPF-based security observability and runtime enforcement tool that monitors process executions, system calls, and I/O activities with Kubernetes awareness. While Tetragon excels in kernel-layer monitoring and security enforcement, its primary focus is on cloud systems and security

observability rather than comprehensive I/O performance analysis across the HPC stack. Finally, Process Monitor for Linux [5] is an eBPF-based monitor that traces system call activity efficiently. It provides a comprehensive list of system calls for observability purposes but is not specifically designed for performance explanation, focusing more on general system activity monitoring.

In summary, while traditional HPC monitoring tools like LMT, Darshan, Recorder, DFTracer, and Score-P have been instrumental in providing performance insights, they often lack the depth to explain underlying causes of performance variability. Emerging eBPF-based tools offer enhanced observability and security features, but they do not fully address the need for detailed, cross-layer performance analysis of I/O stack in HPC environments.

6 Conclusions

We designed DataCrumbs, a flexible and efficient I/O monitoring tool that provides deep visibility into the entire I/O software stack. It covers all layers and libraries, from user-space I/O APIs to kernel-space functions, including both public APIs and internal operations. By decoupling event capture from storage, DataCrumbs efficiently collects detailed performance data with minimal overhead. Our evaluations highlight three key findings: First, DataCrumbs flexibly attaches to multiple I/O software components, capturing over 8,723 functions with a low startup overhead of just 78 ms. Second, it efficiently profiles both user-space and kernel-space I/O operations, incurring only 0.1% profiling overhead, significantly lower than Darshan's 3% overhead. Third, DataCrumbs provides detailed performance data to explain I/O variability, analyze inter-software stack interactions to pinpoint critical optimization layers, and evaluate intra-software bottlenecks by covering all layers and their functions comprehensively. In conclusion, DataCrumbs enables explainable I/O (XIO) analysis by providing comprehensive performance data from HPC storage systems, helping developers and researchers optimize I/O performance at every level of the software stack. We plan to expand DataCrumbs's support to the Lustre filesystem, enabling comprehensive profiling of parallel storage systems on computing facility at Oak Ridge (ORNL) and Argonne National Laboratory.

References

- [1] [n. d.]. BPF CO-RE - eBPF Docs. <https://docs.ebpf.io/concepts/core/>
- [2] [n. d.]. FIO - flexible I/O benchmark. <https://linux.die.net/man/1/fio>
- [3] 2025. Cilium - Cloud Native, eBPF-based Networking, Observability, and Security. <https://cilium.io>
- [4] 2025. Open MPI IO Tuning. <https://github.com/open-mpi/ompi/blob/main/docs/tuning-apps/mpi-io.rst>
- [5] 2025. Process Monitor for Linux. <https://github.com/microsoft/ProcMon-for-Linux> original-date: 2020-07-10T22:58:07Z.
- [6] ALCF. 2020. ALCF Public Data: Argonne Leadership Computing Facility. <https://reports.alcf.anl.gov/data/mira.html>
- [7] Jean Luca Bez, Hammad Ather, and Suren Byna. 2022. Drishti: Guiding End-Users in the I/O Optimization Journey. In *2022 IEEE/ACM International Parallel Systems Workshop (PDSW)*. 1–6.
- [8] Peter Braam. 2019. The Lustre Storage Architecture. doi:10.48550/arXiv.1903.01955 arXiv:1903.01955 [cs].
- [9] Michael J. Brim, Adam T. Moody, Seung-Hwan Lim, Ross Miller, Swen Boehm, Cameron Stanavice, Kathryn M. Mohror, and Sarp Oral. 2023. UnifyFS: A User-level Shared File System for Unified Access to Distributed Local Storage. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. 290–300. doi:10.1109/IPDPS54959.2023.00037 ISSN: 1530-2075.
- [10] S. Byna, Q. Koziol, V. Vishwanath, J. Soumagne, H. Tang, J. Mu, B. Dong, R. A. Warren, F. Tessier, T. Wang, and J. Liu. 2018. Proactive Data Containers (PDC): An Object-centric Data Store for Large-scale Computing Systems. *AGU Fall Meeting Abstracts* 2018 (Dec. 2018), IN34B–09.

- [11] Cyril Cassagnes, Lucian Trestioreanu, Clement Joly, and Radu State. 2020. The rise of eBPF for non-intrusive performance monitoring. In *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*. 1–7. doi:10.1109/NOMS47738.2020.9110434 ISSN: 2374-9709.
- [12] Mugdha Deokar, Jingyang Men, Lucas Castanheira, Ayush Bhardwaj, and Theophilus A. Benson. 2024. An Empirical Study on the Challenges of eBPF Application Development. In *Proceedings of the ACM SIGCOMM 2024 Workshop on eBPF and Kernel Extensions (eBPF '24)*. Association for Computing Machinery, New York, NY, USA, 1–8. doi:10.1145/3672197.3673429
- [13] Hariharan Devarajan, Gerd Heber, and Kathryn Mohror. 2025. H5Intent: Auto-tuning HDF5 With User Intent. *IEEE Transactions on Parallel and Distributed Systems* 36, 2 (2025), 108–119. doi:10.1109/TPDS.2024.3492704
- [14] Hariharan Devarajan and Kathryn Mohror. 2022. Extracting and characterizing I/O behavior of HPC workloads. IEEE, Heidelberg, Germany.
- [15] Hariharan Devarajan, Loic Pottier, Kaushik Velusamy, Huihuo Zheng, Izzet Yildirim, Olga Kogiou, Weikuan Yu, Anthony Kougkas, Xian-He Sun, Jae Seung Yeom, and Kathryn Mohror. 2024. DFTracer: An Analysis-Friendly Data Flow Tracer for AI-Driven Workflows. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, Atlanta, GA.
- [16] Hoang-Dung Do, Valérie Hayot-Sasson, Rafael Ferreira Da Silva, Christopher Steele, Henri Casanova, and Tristan Glataud. 2021. Modeling the Linux page cache for accurate simulation of data-intensive applications. In *2021 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 398–408.
- [17] Dominik Eschweiler, Michael Wagner, Markus Geimer, Knü, Andreas Pfer, Wolfgang E. Nagel, and Felix Wolf. 2012. Open Trace Format 2: The Next Generation of Scalable Trace Formats and Support Libraries. In *Applications, Tools and Techniques on the Road to Exascale Computing*. IOS Press, 481–490. doi:10.3233/978-1-61499-041-3-481
- [18] Todd Gamblin, Matthew LeGendre, Michael R. Collette, Gregory L. Lee, Adam Moody, Bronis R. de Supinski, and Scott Futral. 2015. The Spack package manager: bringing order to HPC software chaos. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*. Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/2807591.2807623
- [19] Jim Garlick. 2025. LMT3 - Lustre Monitoring Tool. <https://github.com/LLNL/lmt> original-date: 2011-03-04T22:06:14Z.
- [20] Bolaji Gbadamosi, Luigi Leonardi, Tobias Pulls, Toke Høiland-Jørgensen, Simone Ferlin-Reiter, Simo Sorce, and Anna Brunström. 2024. The eBPF Runtime in the Linux Kernel. doi:10.48550/arXiv.2410.00026 arXiv:2410.00026 [cs].
- [21] IBM. 2024. mmpmon. <https://www.ibm.com/docs/en/storage-scale/5.2.2?topic=reference-mmpmon-command>
- [22] Florin Isaila, Javier Garcia, Jesus Carretero, Rob Ross, and Dries Kimpe. 2017. Making the case for reforming the I/O software stack of extreme-scale systems. *Advances in Engineering Software* 111 (Sept. 2017), 26–31. doi:10.1016/j.advengsoft.2016.07.003
- [23] Abderaouf Khichane. 2024. *Diagnostic of performance by data interpretation for 5G cloud native network functions*. phdthesis. Université Paris-Saclay. <https://theses.hal.science/tel-04982832>
- [24] Kidney. 2019. 2019 Kidney Tumor Segmentation Challenge. <https://kits19.grand-challenge.org/>
- [25] Andreas Knüpfer, Christian Rössel, Dieter an Mey, Scott Biersdorff, Kai Diethelm, Dominik Eschweiler, Markus Geimer, Michael Gerndt, Daniel Lorenz, Allen Malony, Wolfgang E. Nagel, Yury Oleynik, Peter Philippen, Pavel Saviankou, Dirk Schmidl, Sameer Shende, Ronny Tschüter, Michael Wagner, Bert Wesarg, and Felix Wolf. 2012. Score-P: A Joint Performance Measurement Run-Time Infrastructure for Periscope, Scalasca, TAU, and Vampir. In *Tools for High Performance Computing 2011*, Holger Brunst, Matthias S. Müller, Wolfgang E. Nagel, and Michael M. Resch (Eds.). Springer, Berlin, Heidelberg, 79–91. doi:10.1007/978-3-642-31476-6_7
- [26] Anthony Kougkas, Hariharan Devarajan, and Xian-He Sun. 2018. Hermes: a heterogeneous-aware multi-tiered distributed I/O buffering system. In *Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '18)*. Association for Computing Machinery, New York, NY, USA, 219–230. doi:10.1145/3208040.3208059
- [27] Julian Kunkel. 2025. HPC IO Benchmark v 4.0.0. <https://github.com/hpc/ior/releases/tag/4.0.0> original-date: 2015-06-23T15:56:39Z.
- [28] Jianwei Li, Wei-keng Liao, Alok Choudhary, Robert Ross, Rajeev Thakur, William Gropp, Rob Latham, Andrew Siegel, Brad Gallagher, and Michael Zingale. 2003. Parallel NetCDF: A High-Performance Scientific I/O Interface. In *Proceedings of the 2003 ACM/IEEE Conference on Supercomputing (Phoenix, AZ, USA) (SC '03)*. Association for Computing Machinery, New York, NY, USA, 39. doi:10.1145/1048935.1050189
- [29] Glenn K. Lockwood, Nicholas J. Wright, Shane Snyder, Philip Carns, George Brown, and Kevin Harms. 2018. TOKIO on ClusterStor: Connecting Standard Tools to Enable Holistic I/O Performance Analysis. (2018). <https://escholarship.org/uc/item/8j14j182>
- [30] Jay Lofstead, Fang Zheng, Scott Klasky, and Karsten Schwan. 2008. Input/output apis and data organization for high performance scientific computing. In *2008 3rd Petascale Data Storage Workshop*. IEEE, 1–6.
- [31] Avantika Mathur, Mingming Cao, Suparna Bhattacharya, Andreas Dilger, Alex Tomas, and Laurent Vivier. 2007. The new ext4 filesystem: current status and future plans. In *Proceedings of the Linux symposium*, Vol. 2. Citeseer, 21–33.
- [32] Amrita Mathuriya, Deborah Bard, Peter Mendygral, Lawrence Meadows, James Arnemann, Lei Shao, Siyu He, Tuomas Kärmä, Diana Moise, Simon J. Pennycook, Kristyn Maschhoff, Jason Sewall, Nalini Kumar, Shirley Ho, Michael F. Ringenburt, Prabhat Prabhat, and Victor Lee. 2018. CosmoFlow: Using Deep Learning to Learn the Universe at Scale. In *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*. 819–829. doi:10.1109/SC.2018.00068
- [33] Nolan Monnier, Jay Lofstead, Margaret Lawson, and Matthew Curry. 2019. Profiling platform storage using io500 and mistral. In *2019 IEEE/ACM Fourth International Parallel Data Systems Workshop (PDSW)*. IEEE, 60–73.
- [34] Adam Moody, Greg Bronevetsky, Kathryn Mohror, and Bronis R. de Supinski. 2010. Design, Modeling, and Evaluation of a Scalable Multi-level Checkpointing System. In *SC '10: Proceedings of the 2010 ACM/IEEE International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–11. doi:10.1109/SC.2010.18 ISSN: 2167-4337.
- [35] Mohammed Islam Naas, François Trahay, Alexis Colin, Pierre Olivier, Stéphane Rubini, Frank Singhoff, and Jalil Boukhobza. 2021. EZIOTracer: unifying kernel and user space I/O tracing for data-intensive applications. In *Proceedings of the Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems*. 1–11.
- [36] Andrii Nakryiko. [n. d.]. BPF CO-RE reference guide. <https://nakryiko.com/posts/bpf-core-reference-guide/>
- [37] NERSC. 2024. Darshan I/O Profiler - NERSC Documentation. <https://docs.nersc.gov/tools/performance/darshan/>
- [38] Dan Oh. 2012. System level jitter characterization of high speed I/O systems. In *2012 IEEE International Symposium on Electromagnetic Compatibility*. IEEE, 173–178.
- [39] OLCF. 2025. Data Storage and Transfers — OLCF User Documentation. <https://docs.olcf.ornl.gov/data/index.html#darshan-runtime-and-i-o-profiling>
- [40] Arnab K Paul, Ryan Chard, Kyle Chard, Steven Tuecke, Ali R Butt, and Ian Foster. 2019. Fsmontor: Scalable file system monitoring for arbitrary storage systems. In *2019 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE, 1–11.
- [41] Alastair Robertson. 2025. bpftrace: High-level tracing language for Linux. <https://github.com/bpftrace/bpftrace> original-date: 2018-08-31T04:34:44Z.
- [42] Matthew Rocklin. 2015. Dask: Parallel Computation with Blocked algorithms and Task Scheduling. 126–132. doi:10.25080/Majors-7b98e3ed-013
- [43] M. Rynge, G. Juve, J. Kinney, J. Good, B. Berriman, A. Merrihew, and E. Deelman. 2014. Producing an Infrared Multiwavelength Galactic Plane Atlas Using Montage, Pegasus, and Amazon Web Services. *Annual Astronomical Data Analysis Software and Systems (ADAS)* 485 (May 2014), 211. <https://ui.adsabs.harvard.edu/abs/2014ASPC..485..211R> Conference Name: Astronomical Data Analysis Software and Systems XXIII ADS Bibcode: 2014ASPC..485..211R.
- [44] Frank Schmuck and Roger Haskin. 2002. GPFS: A Shared-Disk File System for Large Computing Clusters. <https://shorturl.at/yLV6P>
- [45] Hongzhang Shan, Katie Antypas, and John Shalf. 2008. Characterizing and predicting the I/O performance of HPC applications using a parameterized synthetic benchmark. In *SC '08: Proceedings of the 2008 ACM/IEEE Conference on Supercomputing*. 1–12. doi:10.1109/SC.2008.5222721 ISSN: 2167-4337.
- [46] Nayanah Siva. 2008. 1000 genomes project. *Nature Biotechnology* 26, 3 (March 2008), 256–257. <https://github.com/pegasus-isi/1000genome-workflow> Publisher: Nature Publishing Group.
- [47] Shane Snyder, Philip Carns, Kevin Harms, Robert Ross, Glenn K. Lockwood, and Nicholas J. Wright. 2016. Modular HPC I/O Characterization with Darshan. In *2016 5th Workshop on Extreme-Scale Programming Tools (ESPT)*. 9–17. doi:10.1109/ESPT.2016.006
- [48] David Soldani, Petrit Nahi, Hani Bour, Saber Jafarizadeh, Mohammed F. Soliman, Leonardo Di Giovanna, Francesco Monaco, Giuseppe Ognibene, and Fulvio Risso. 2023. eBPF: A New Approach to Cloud-Native Observability, Networking and Security for Current (5G) and Future Mobile Networks (6G and Beyond). *IEEE Access* 11 (2023), 57174–57202. doi:10.1109/ACCESS.2023.3281480 Conference Name: IEEE Access.
- [49] Tetragon. 2024. Tetragon - eBPF-based Security Observability and Runtime Enforcement. <https://tetragon.io/>
- [50] Rajeev Thakur, William Gropp, and Ewing Lusk. 1999. On implementing MPI-IO portably and with high performance. In *Proceedings of the sixth workshop on I/O in parallel and distributed systems (IOPADS '99)*. Association for Computing Machinery, New York, NY, USA, 23–32. doi:10.1145/301816.301826
- [51] The HDF Group. 1997–2022. Hierarchical Data Format, version 5. <https://www.hdfgroup.org/HDF5/>.
- [52] François Trahay, François Rue, Mathieu Faverge, Yutaka Ishikawa, Raymond Namyst, and Jack Dongarra. 2011. EZTrace: A Generic Framework for Performance Analysis. In *2011 11th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*. 618–619. doi:10.1109/CCGrid.2011.83

- [53] IO Visor. 2025. BCC - Tools for BPF-based Linux IO analysis, networking, monitoring, and more. <https://github.com/iovisor/bcc> original-date: 2015-05-01T19:52:32Z.
- [54] Chen Wang, Kathryn Mohror, and Marc Snir. 2021. File System Semantics Requirements of HPC Applications. In *Proceedings of the 30th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '21)*. Association for Computing Machinery, New York, NY, USA, 19–30. doi:10.1145/3431379.3460637
- [55] Chen Wang, Kathryn Mohror, and Marc Snir. 2024. Formal Definitions and Performance Comparison of Consistency Models for Parallel File Systems. *IEEE Transactions on Parallel and Distributed Systems* (2024). Publisher: IEEE.
- [56] Chen Wang, Jinghan Sun, Marc Snir, Kathryn Mohror, and Elsa Gonsiorowski. 2020. Recorder 2.0: Efficient Parallel I/O Tracing and Analysis. In *2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 1–8. doi:10.1109/IPDPSW50202.2020.00176
- [57] Chen Wang, Izzet Yildirim, Hariharan Devarajan, Kathryn Mohror, and Marc Snir. 2025. Recorder: Comprehensive Parallel I/O Tracing and Analysis. *arXiv preprint arXiv:2501.04654* (2025).
- [58] Cong Xu, Shane Snyder, Omkar Kulkarni, Vishwanath Venkatesan, Philip Carns, Suren Byna, Robert Sisneros, and Kalyana Chandalavada. [n. d.]. DXT: Darshan eXtended Tracing. ([n. d.]).
- [59] Izzet Yildirim, Hariharan Devarajan, Anthony Kougkas, Xian-He Sun, and Kathryn Mohror. 2023. IOMax: Maximizing Out-of-Core I/O Analysis Performance on HPC Systems. In *Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis (SC-W 2023), November 12–17, 2023, Denver, CO, USA*. doi:10.1145/3624062.3624191
- [60] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, and Andrew Quinn. 2023. bpftime: userspace eBPF Runtime for Uprobe, Syscall and Kernel-User Interactions. doi:10.48550/arXiv.2311.07923 arXiv:2311.07923 [cs].